Errand Agent — design note

v3 · 8 Nov 2026. The PDF is authoritative and the eight tool contracts are unchanged. Details, trade-offs and tests: DECISIONS.md.

Shape. CLI → local HTTP API → explicit workflow → `ToolClient` → in-process mocks; PostgreSQL 16 + pgvector holds all state; stateless workers run queued jobs. One LLM, reached through the user's local ModelMux gateway (Claude or Codex; the models themselves run on the providers' servers, DECISIONS §17), extracts language and proposes actions. Code owns dates, ranking, prices, IDs, authorization, execution and every consequential message. *Trade-off:* a single agent plus deterministic workers is easier to bound and test than several LLM agents, at the cost of less flexible phrasing.

State model. Each session is an explicit, CAS-versioned state machine: `collecting → comparing → held → awaiting_approval → executing → verifying → completed`, plus `retry_wait`, `pending_verification`, `conflict` and `failed`. Each state allows only certain tools, and every ID argument is an enum of real values. A correction keeps unrelated facts. Model output is validated, repaired at most 3 times, then a numbered menu takes over.

Approval enforcement. Code renders an immutable card (action, provider, slot, price, hold, expiry). Only `yes` or `confirm <token>`, citing the latest card actually displayed, is accepted. Any material change supersedes the card. The model's `book`/`cancel`/`reschedule` call must match the approved snapshot exactly, and code dispatches the snapshot, never the model's arguments.

Idempotency and recovery. One idempotency key per approved operation; each dispatch is a durable, fenced attempt reserved before the call. A timeout or crash always forces `get_booking_status` (or `list_bookings`) before any same-key retry. A validated dispatch is backed by a claimed job, so a worker finishes it after a crash. Lifetime caps (3 dispatches, 6 checks, 40 model calls, 300 s) survive restarts. *Trade-off:* Postgres locks and fenced claims add machinery, but give real concurrency and crash tests instead of a singleton process.

Memory. Typed facts (area, earliest time, provider per service, notes) are explicit (user said so) or inferred (code, from 3 completed bookings; unused until confirmed). Inspect/correct/forget are code-handled commands. Forget erases value and vector together. Retrieval hard-filters in SQL (user, confirmed, current model, scope), then ranks by exact cosine (local nomic-embed, threshold calibrated to 0.45). The context cap is 4,096 bytes per request; reviews are dropped first, then low-ranked memories. Prior chat never reaches the model.

Observability. Every run traces state transitions, exact model context, every attempt, tools with arguments and sanitized results, latency, token usage and cost ("unavailable": subscription-billed). Proactive cancellations, retries and outcomes reach the user through an outbox. Raw JSONL: `artifacts/traces/timeout-after-success-…jsonl`. Real run (Codex), annotated:

```
38 operation.created     key op-16e13a49…  (approval consumed, revision 1)
41 model.call act        proposes book(HLD-000001, op-16e13a49…)  5.3 s, 4,736 prompt tokens
42 guardrail.snapshot_check  accepted: matches the approved card exactly
43 operation.dispatch_reserved  attempt 1 reserved before the call
45 tool.call book        error ToolTimeoutError, effect "unknown"   ← booked, but the reply was lost
47 state.transition      executing → verifying  (no success claimed, no blind retry)
49 tool.call get_booking_status(op-16e13a49…) → booked BK-000002  ← source of truth
52 operation.finished    succeeded, verified: true → "Booked and verified … BK-000002"
```